

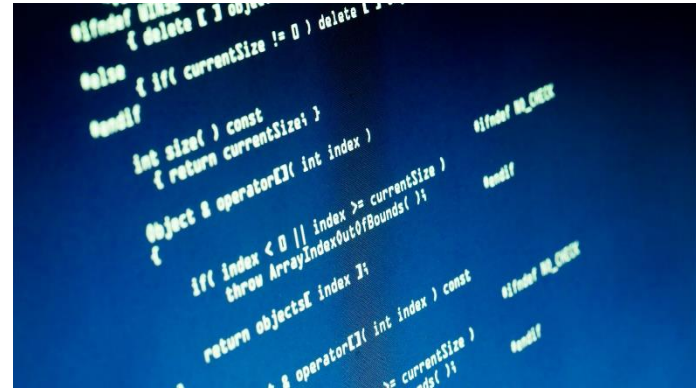
Tehnici de programare - TP



Cursul 9 – Liste

Ș.l. dr. ing. Cătălin Iapă

catalin.iapa@cs.upt.ro



De data trecută

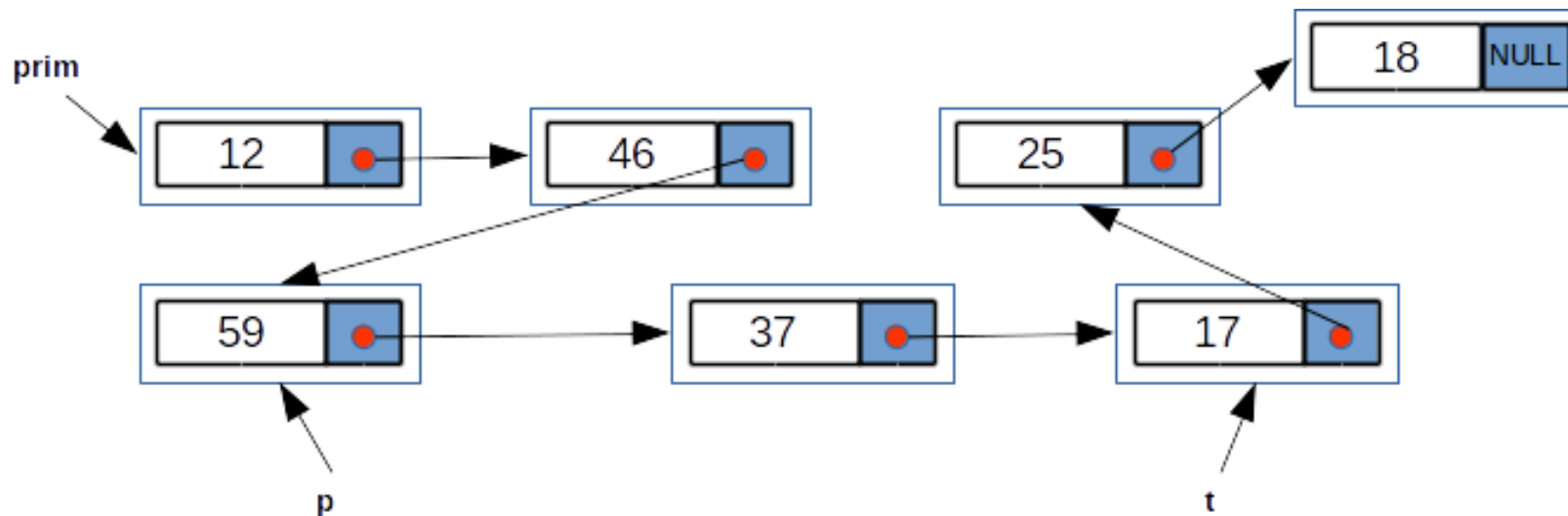
Pointeri – recapitulare

Transmiterea prin dublu pointer

Coada, Stiva implementate cu liste

Liste multiplu înlănțuite

Relații între nodurile unei liste



prim este adresa elementului cu valoarea 12;

prim->info==12

prim->urm->info==46

prim->urm este adresa elementului cu valoarea 46

prim->urm->urm==p

p->info==59

Liste simplu înlănțuite - nodul

// un element al listei – un nod

```
typedef struct elem{
```

```
    int info;
```

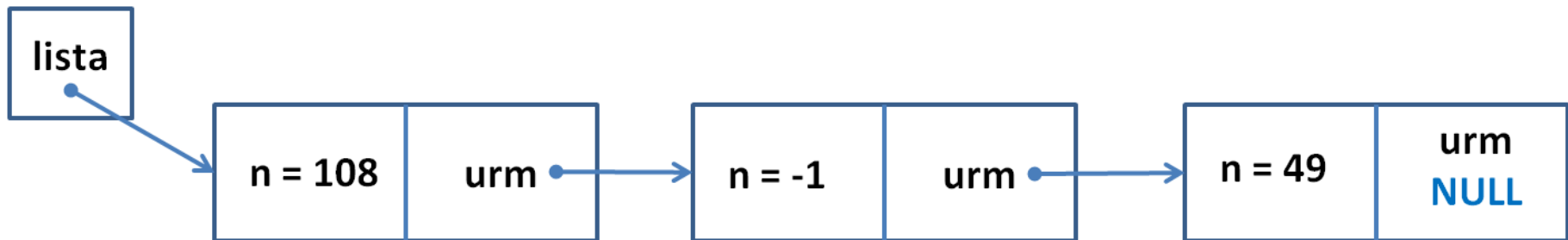
// informatia utila

```
    struct elem *urm;
```

// urmatorul nod

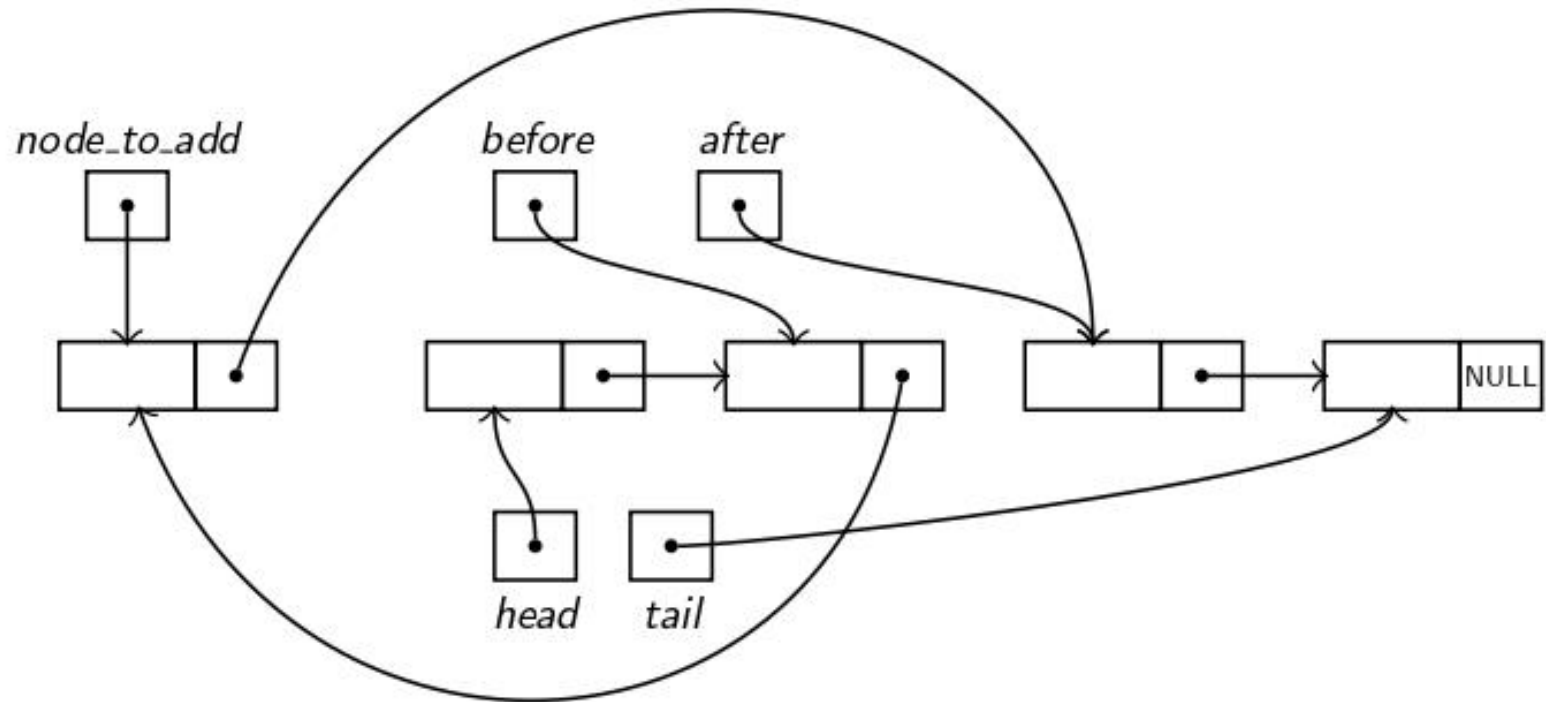
```
}elem;
```

```
elem *lista = NULL;
```



Ordered Lists

Adding a Node to an Ordered List



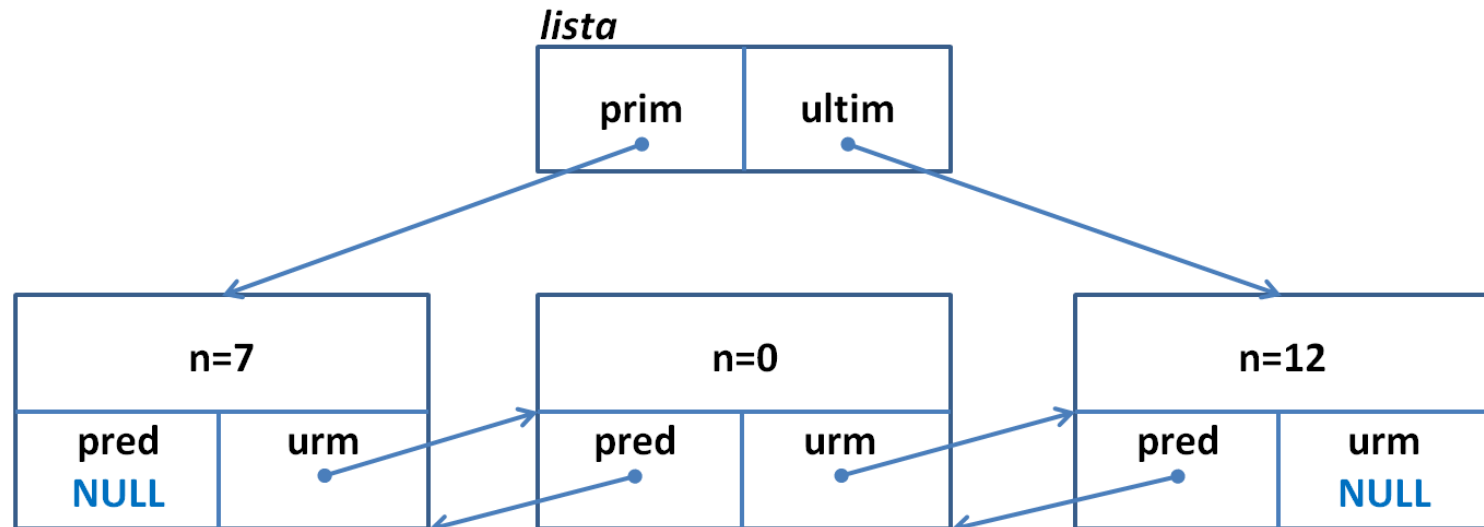
Liste liniare dublu înlănțuite

Pe lângă listele simplu înlănțuite, în C putem folosi și **liste liniare dublu înlănțuite**.

Acestea sunt similare cu listele simplu înlănțuite, dar fiecare nod are și **un pointer către nodul anterior**, permițând **parcurgerea listei în ambele direcții**.

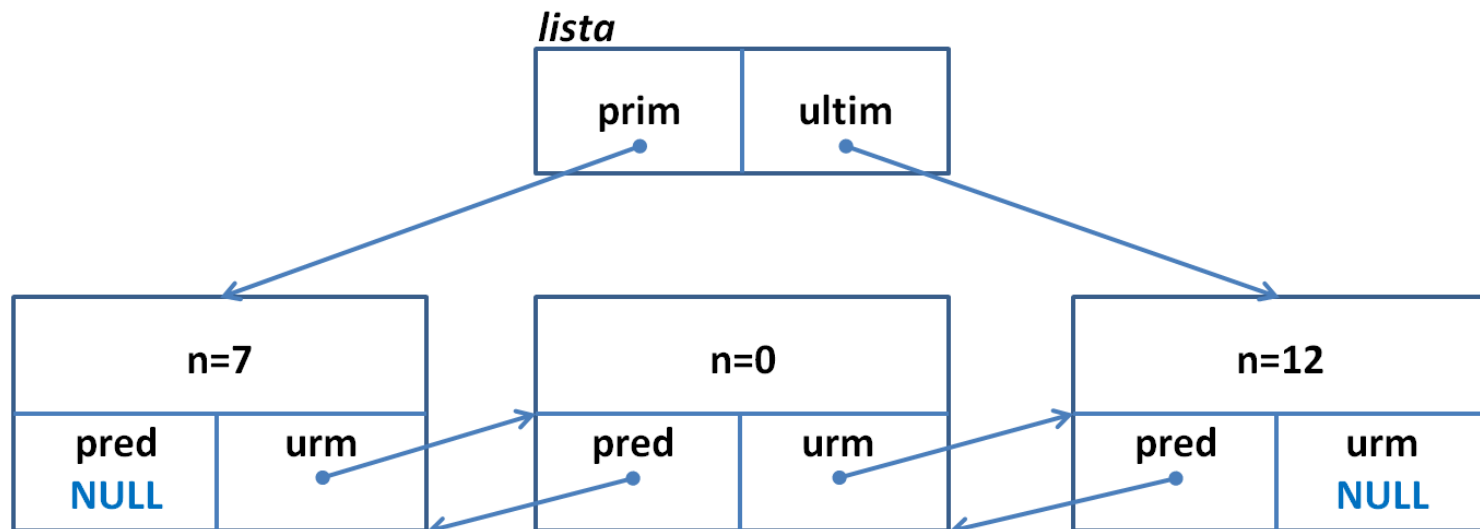
Structura

```
typedef struct nod{  
    int data;  
    struct nod *pred;  
    struct nod *urm;  
}nod;
```



Nodurile prim și ultim

```
typedef struct{  
    nod *prim;    // primul nod din lista  
    nod *ultim;   // ultimul nod din lista  
}Lista;
```



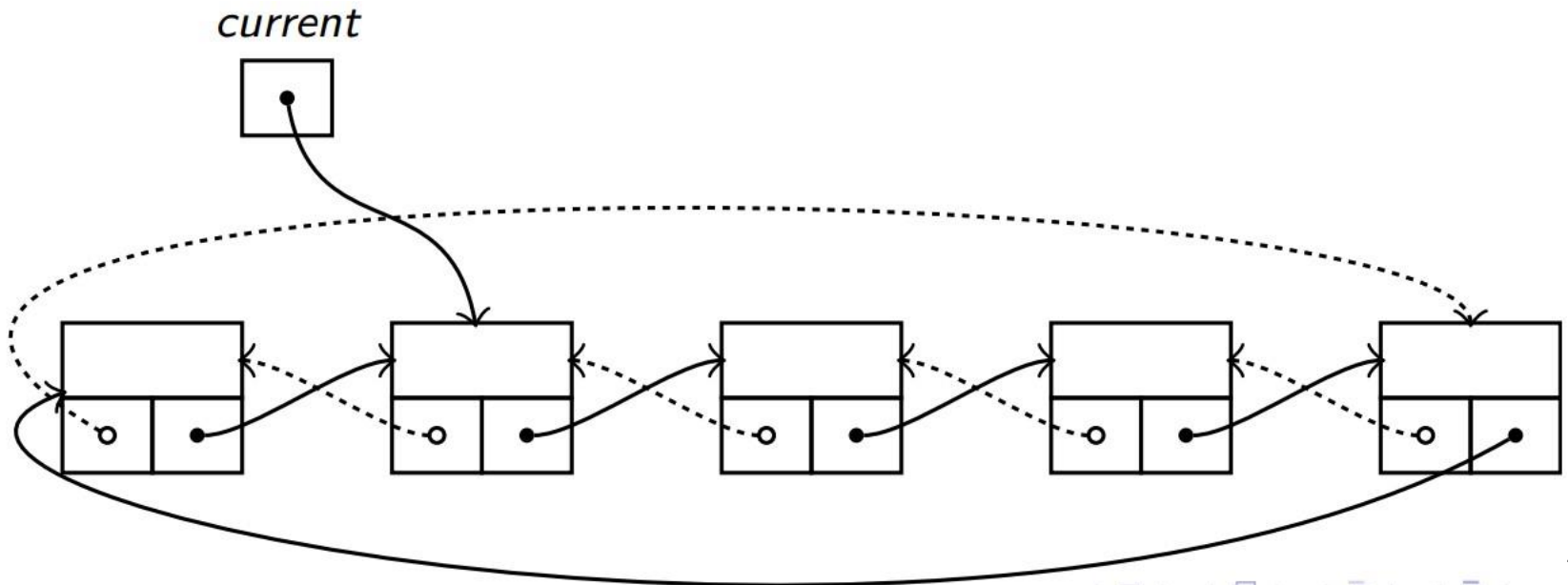
Liste simplu sau dublu înlănțuite?

Pentru **liste scurte**, de doar câteva elemente, timpul necesar pentru iterarea unei **liste simplu înlănțuite** este compensat de timpul mai mic necesar pentru alte operații, deoarece acestea sunt mai simple decât la listele dublu înlănțuite.

Dacă în schimb **lista trece de un anumit număr de elemente** și **operațiile de ștergere sau inserare înainte de elementul curent sunt frecvente**, atunci deja diferența între cele două tipuri de date devine semnificativă.

Liste circulare

Listele dublu înlanțuite se pot organiza ușor ca **liste circulare** (în care nu există nici un element prim sau ultim ci **doar un pointer curent spre orice nod al listei** și în care nu avem **NULL** la nici un **pointer pred sau urm al nodurilor**).



Circular Doubly Linked Lists

```
typedef struct _node {  
    int info;  
    struct _node *prev;  
    struct _node *next;  
} node;
```

- If we put a dummy sentinel node into the list, so that it never gets empty, then all operations upon the list are simplified (there will be no special cases to handle).

```
void init_list(node ** start)  
{  
    *start = create_node(-1);    /* Create the sentinel  
                                * node. */  
  
    /*  
    * Link both pointers of the sentinel node back to  
    * itself (circular doubly linked list with one node).  
    */  
    (*start)->prev = (*start)->next = *start;  
}
```

Circular Doubly Linked Lists

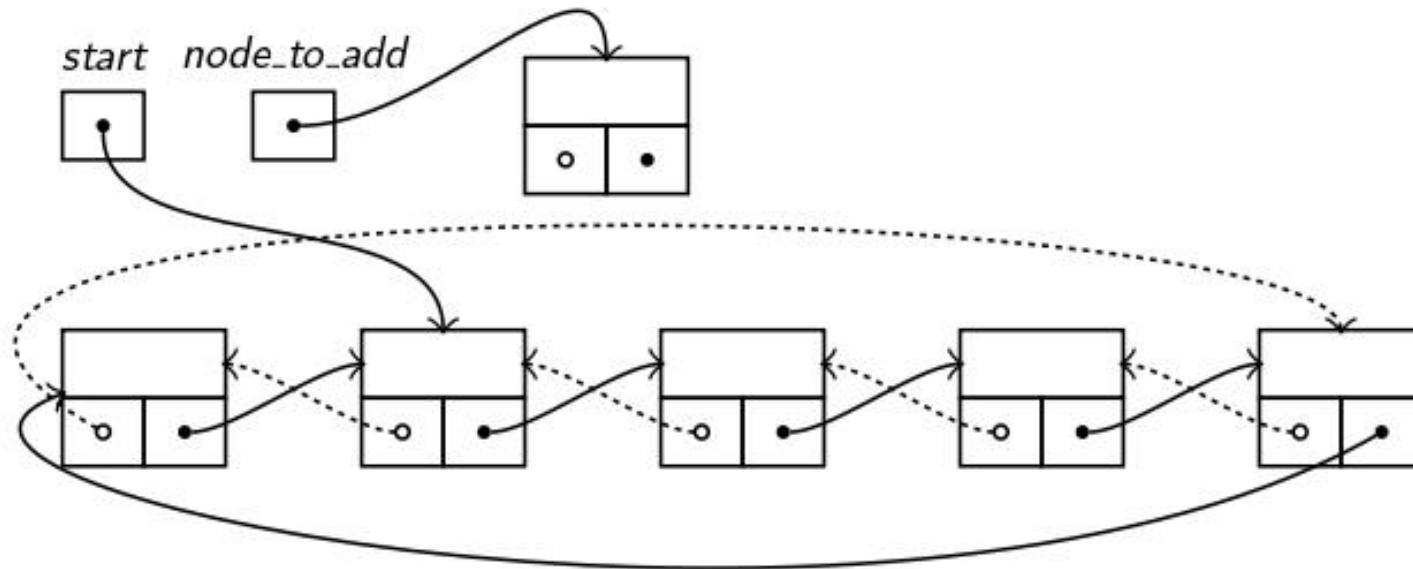
Visiting the Nodes in a Circular Doubly Linked List

```
void print_list(node * start)
{
    node *p = start->next;           /* Start from right after
                                     * the sentinel. */

    /*
     * If the next node after the sentinel is the sentinel,
     * then the list is actually empty.
     */
    if (p == start)
        printf("The list is empty.\n");
    else {
        /*
         * Loop while the sentinel is not reached again.
         */
        while (p != start) {
            printf("%d_", p->info);
            p = p->next;
        }
        printf("\n");
    }
}
```

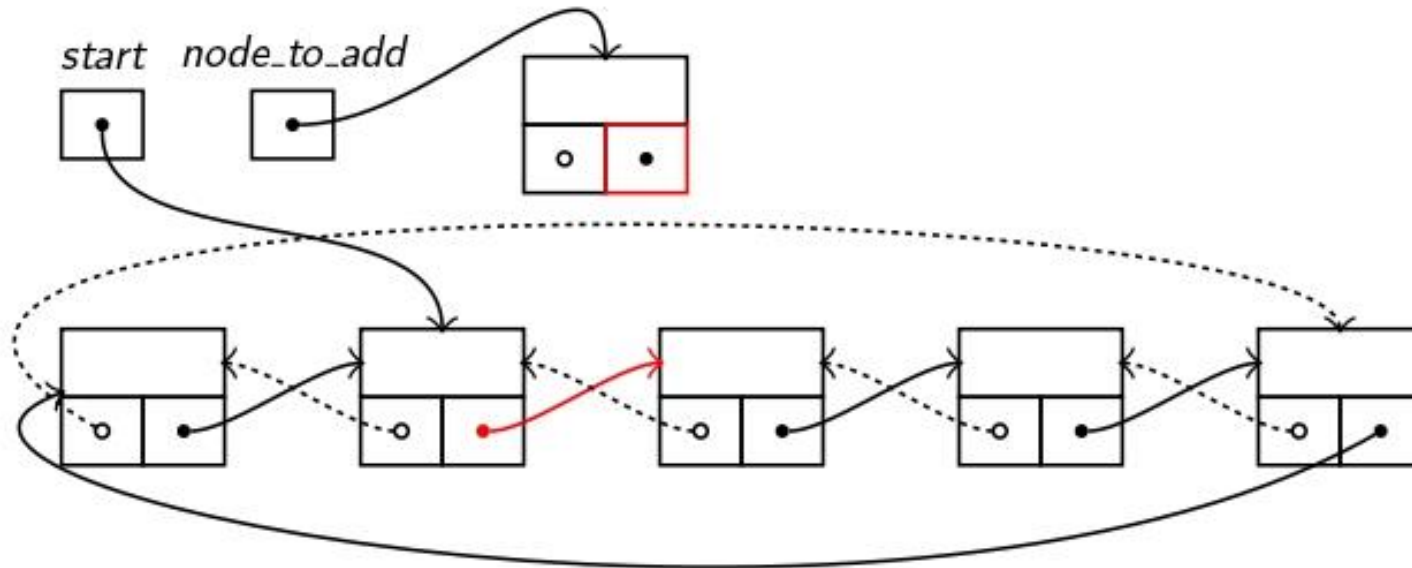
Doubly Linked Lists

- Adding a node to a doubly linked circular list



Doubly Linked Lists

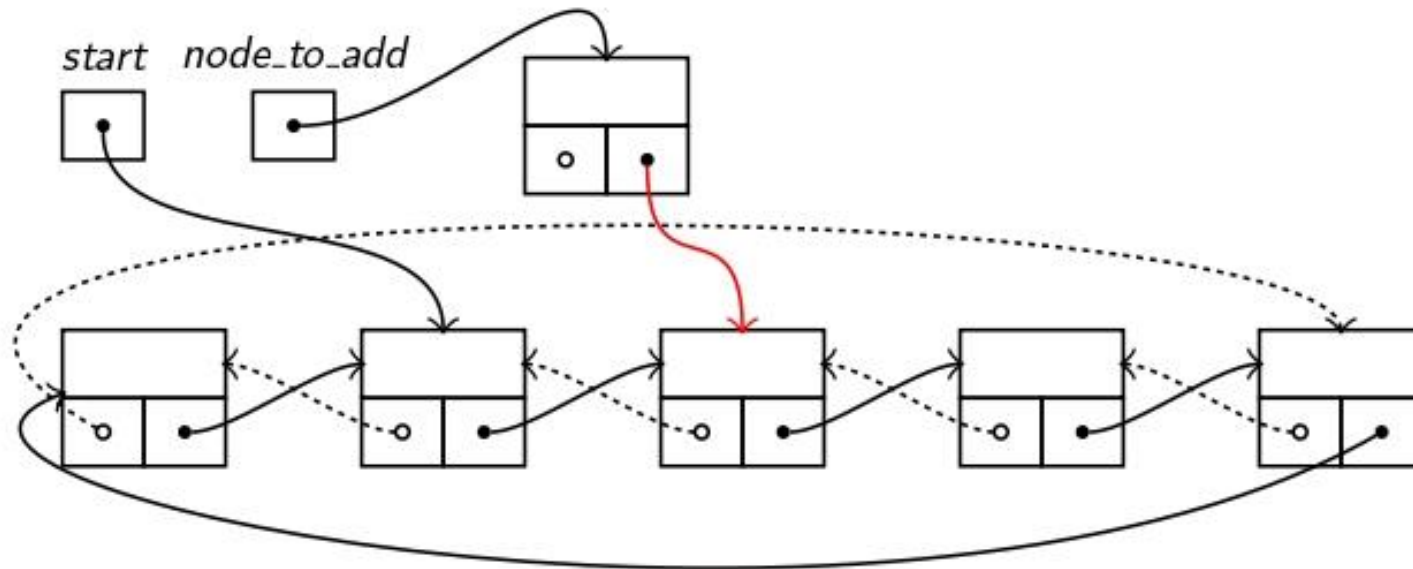
- Adding a node to a doubly linked circular list



```
node_to_add→next = start→next;
```

Doubly Linked Lists

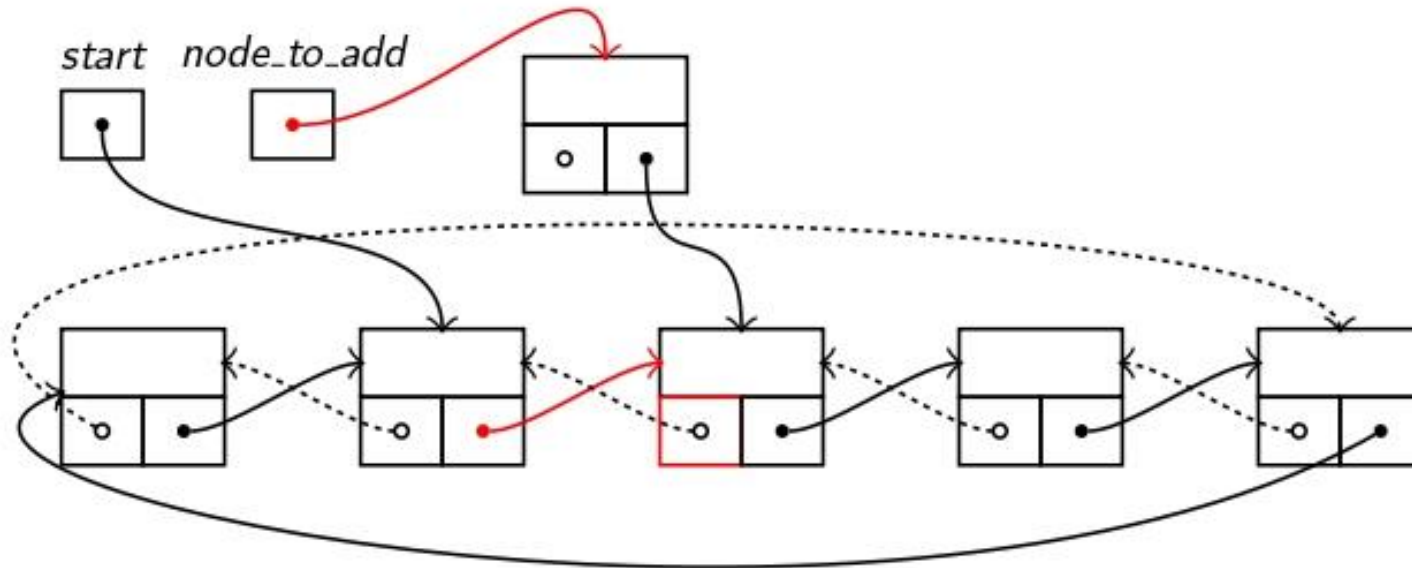
- Adding a node to a doubly linked circular list



```
node_to_add -> next = start -> next;
```

Doubly Linked Lists

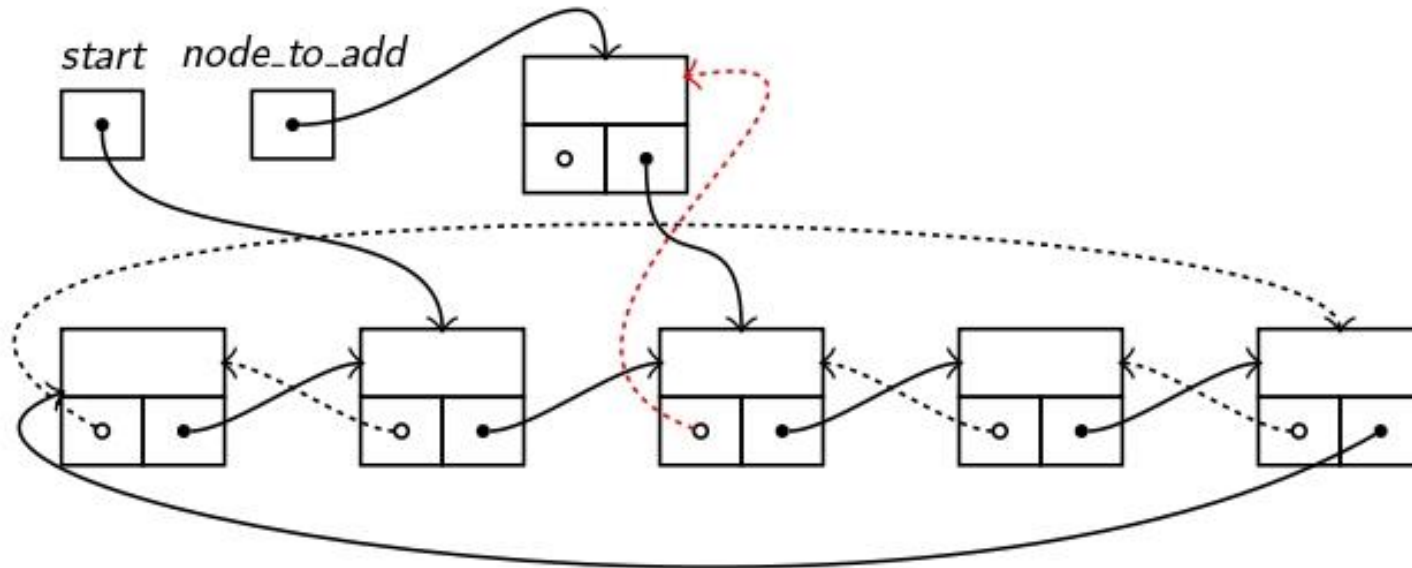
- Adding a node to a doubly linked circular list



```
start -> next -> prev = node_to_add;
```


Doubly Linked Lists

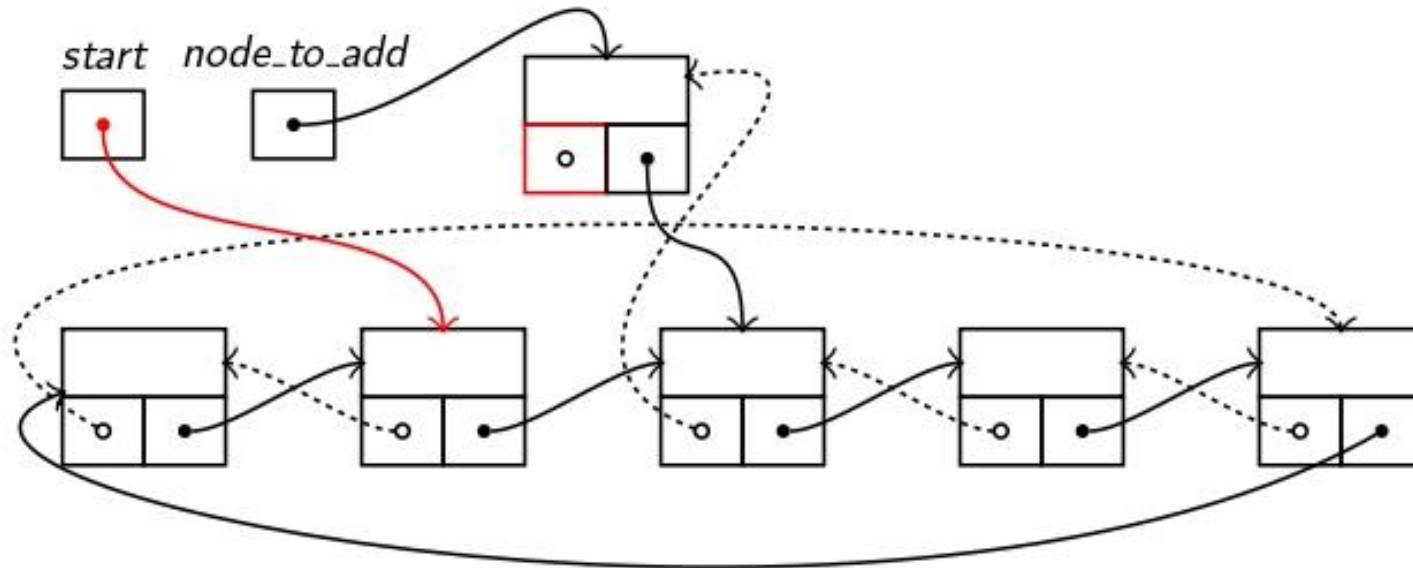
- Adding a node to a doubly linked circular list



```
start → next → prev = node_to_add;
```

Doubly Linked Lists

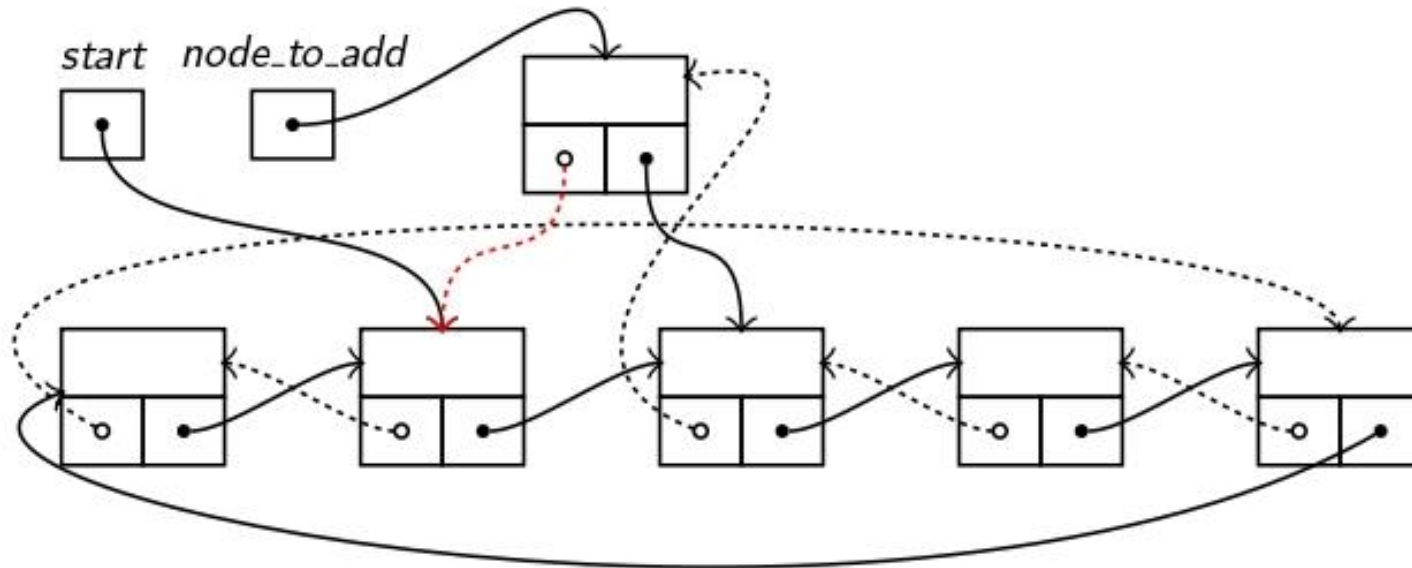
- Adding a node to a doubly linked circular list



```
node_to_add -> prev = start ;
```

Doubly Linked Lists

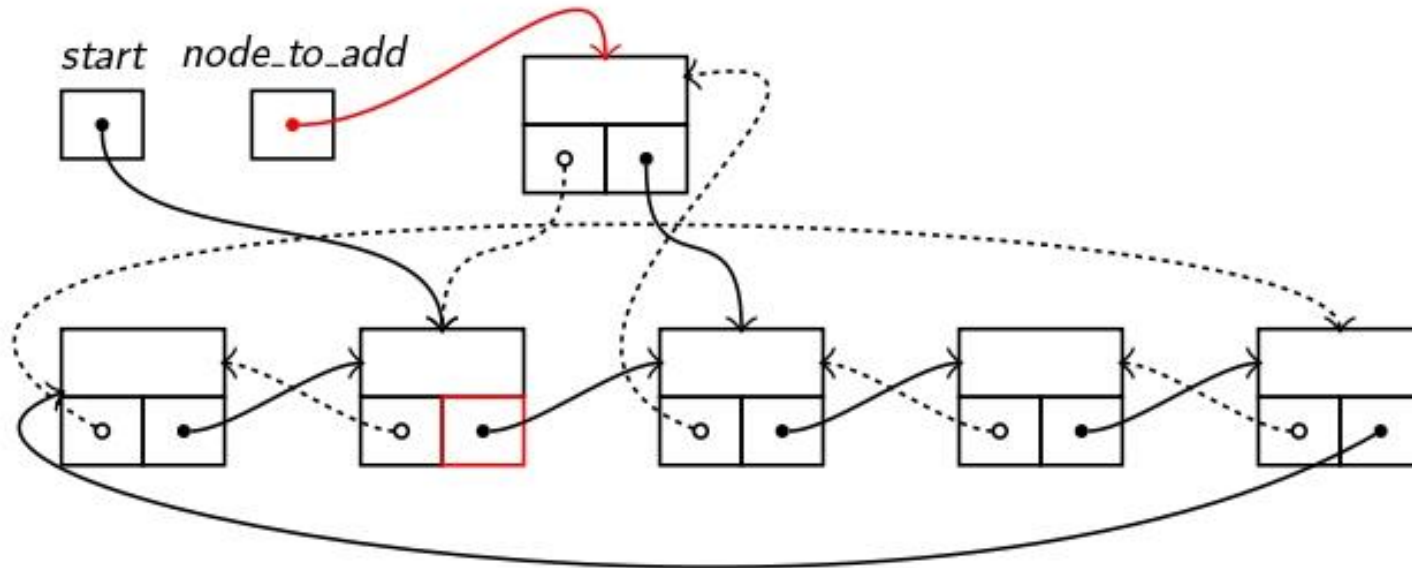
- Adding a node to a doubly linked circular list



```
node_to_add -> prev = start ;
```

Doubly Linked Lists

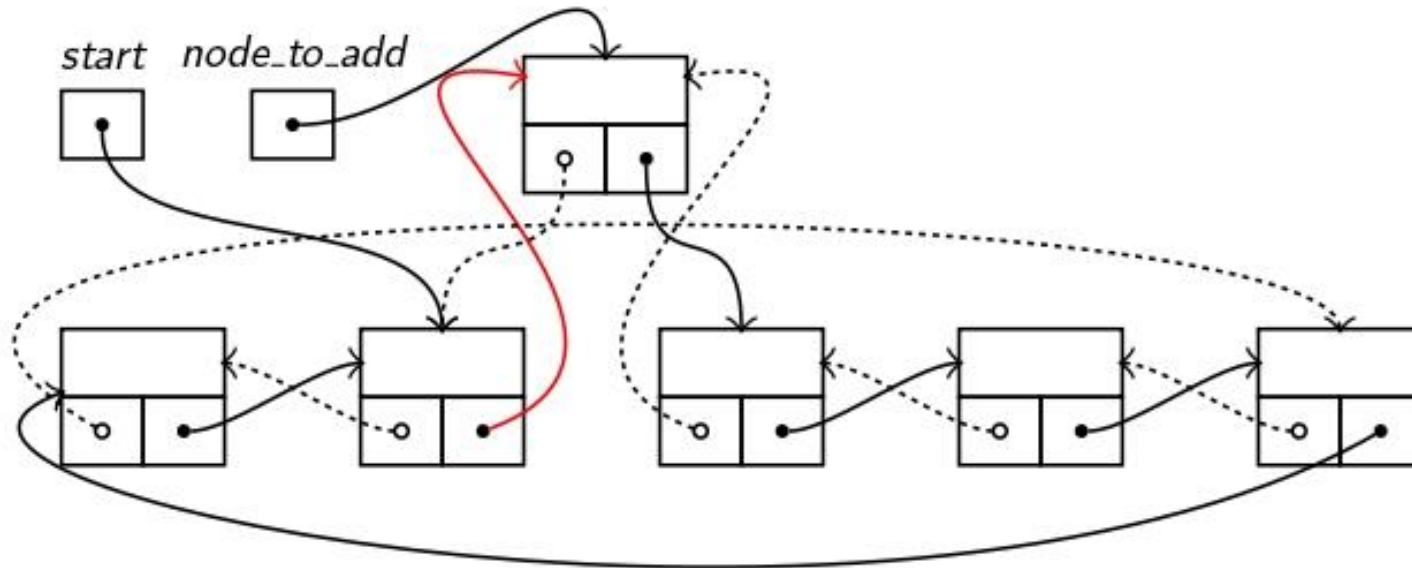
- Adding a node to a doubly linked circular list



```
start -> next = node_to_add;
```

Doubly Linked Lists

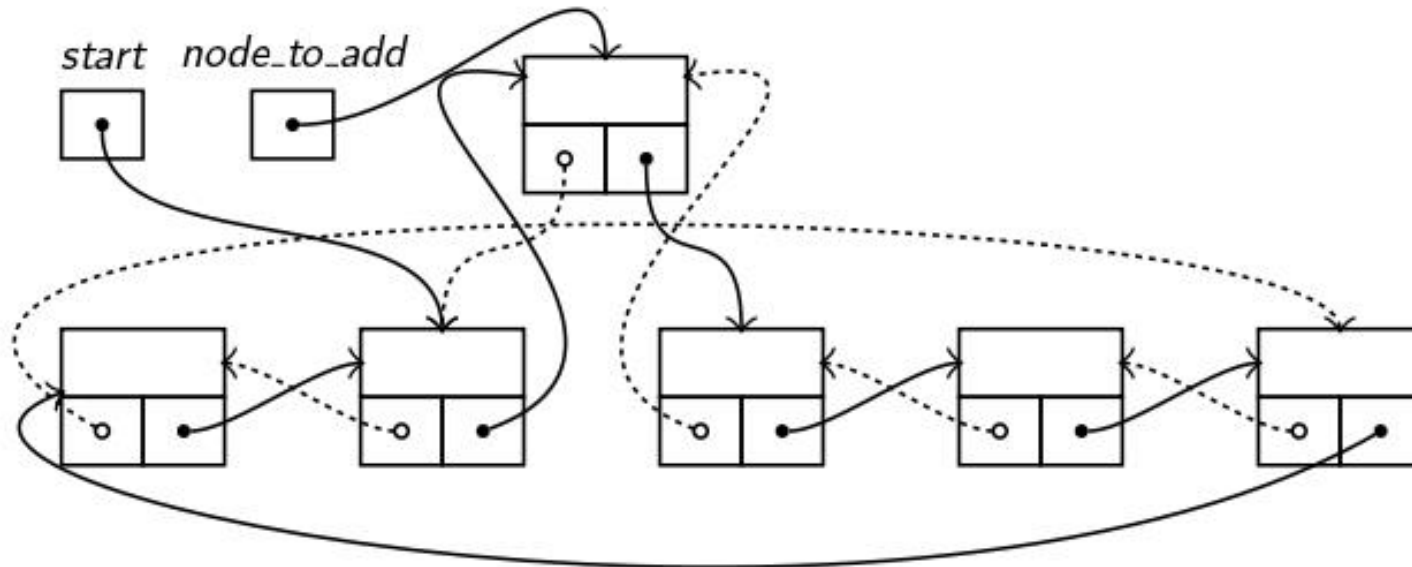
- Adding a node to a doubly linked circular list



```
start -> next = node_to_add;
```

Doubly Linked Lists

- Adding a node to a doubly linked circular list



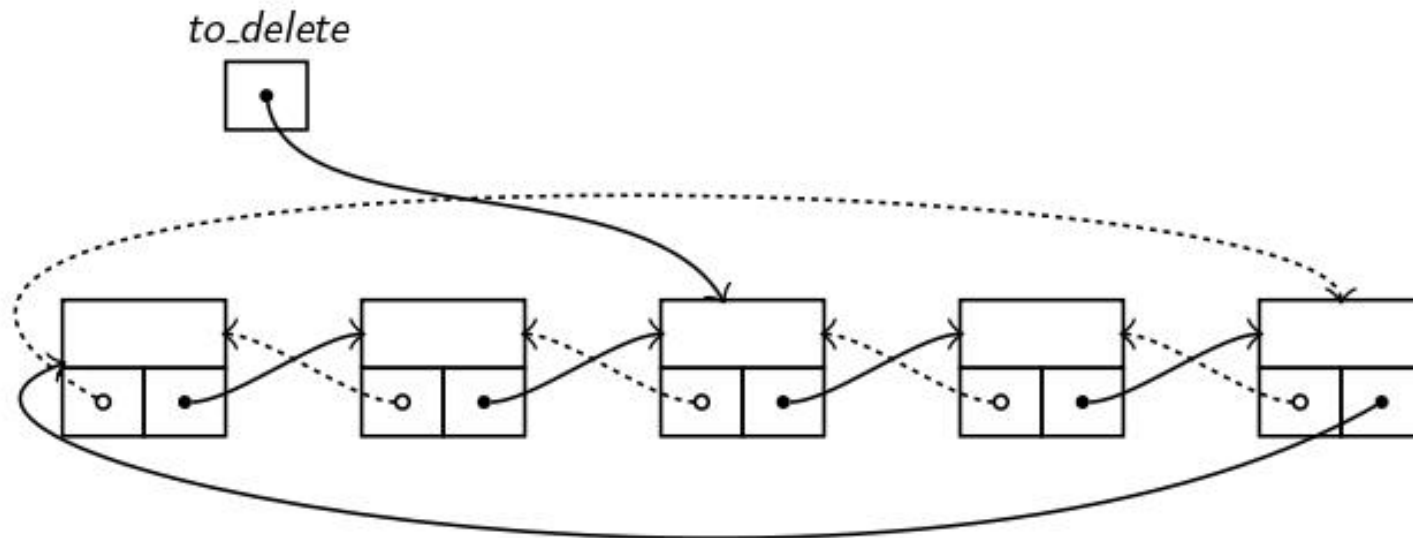
Circular Doubly Linked Lists

Adding a Node to a Circular Doubly Linked List

```
void add_node(node * node_to_add, node * start)
{
    /*
     * Make the double link between the new node and the
     * node after the sentinel
     */
    node_to_add->next = start->next;
    start->next->prev = node_to_add;
    /*
     * Make the double link between the new node and the
     * sentinel.
     */
    node_to_add->prev = start;
    start->next = node_to_add;
}
```

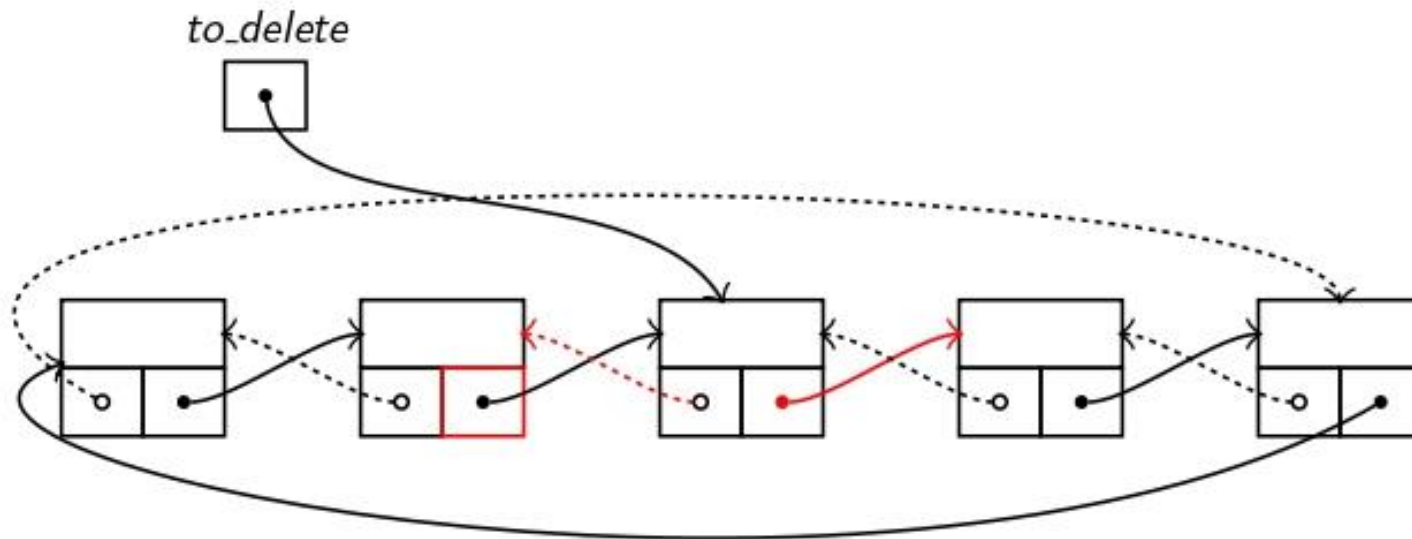
Doubly Linked Lists

- Deleting a node from a doubly linked circular list



Doubly Linked Lists

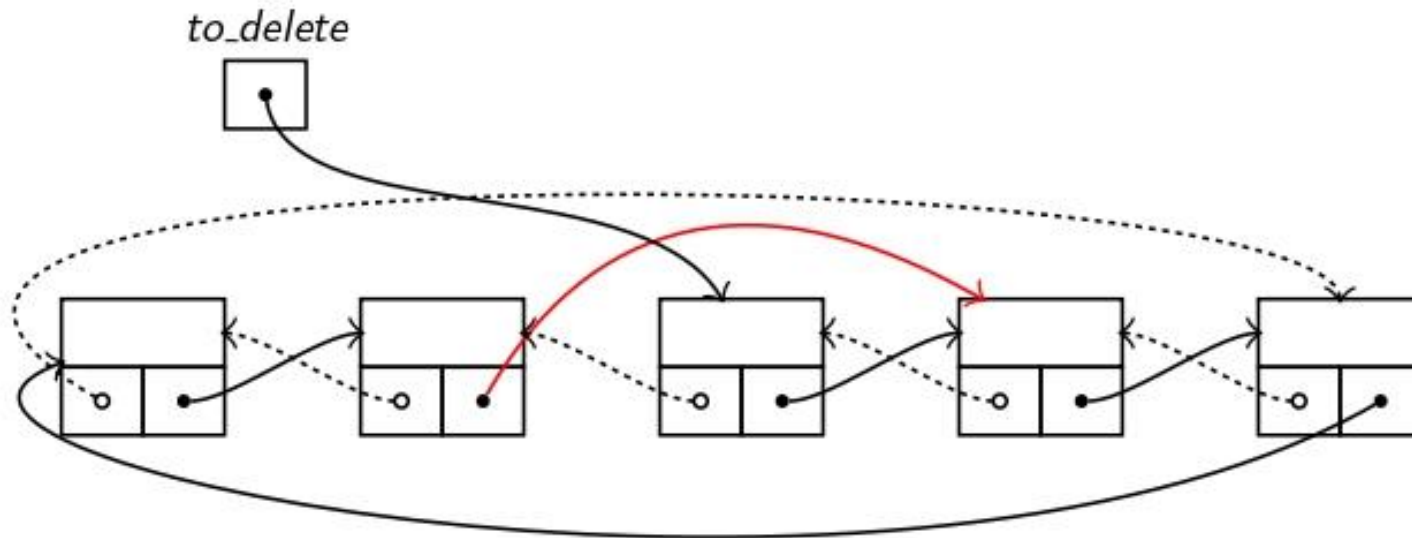
- Deleting a node from a doubly linked circular list



```
to_delete -> prev -> next = to_delete -> next;
```

Doubly Linked Lists

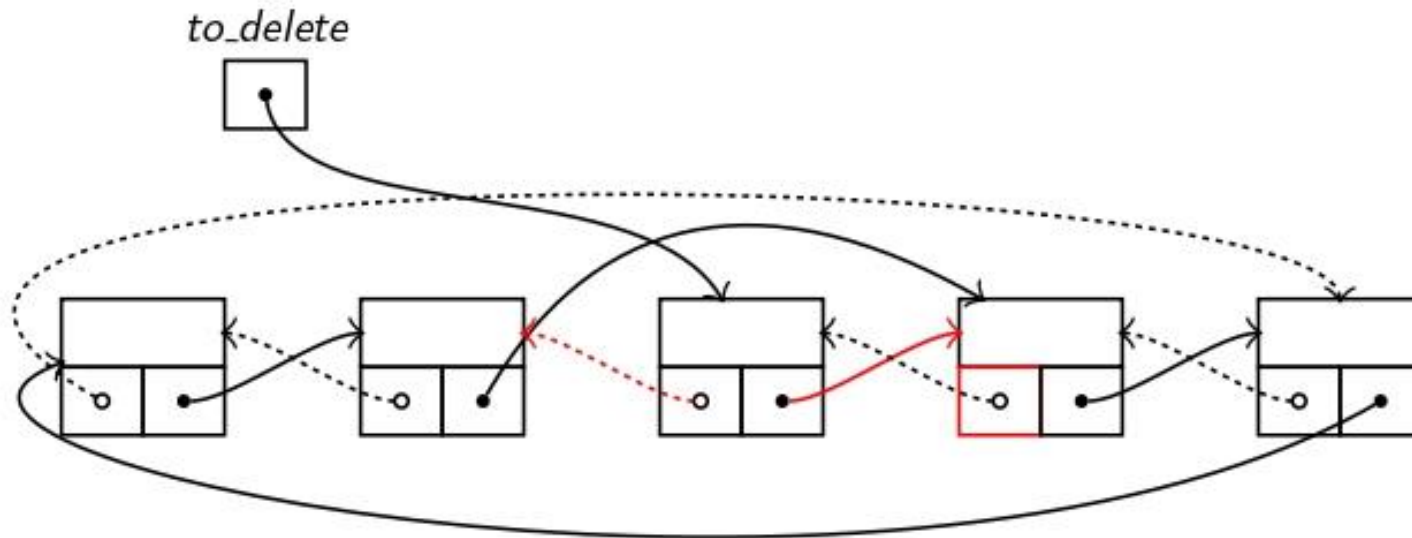
- Deleting a node from a doubly linked circular list



```
to_delete -> prev -> next = to_delete -> next;
```

Doubly Linked Lists

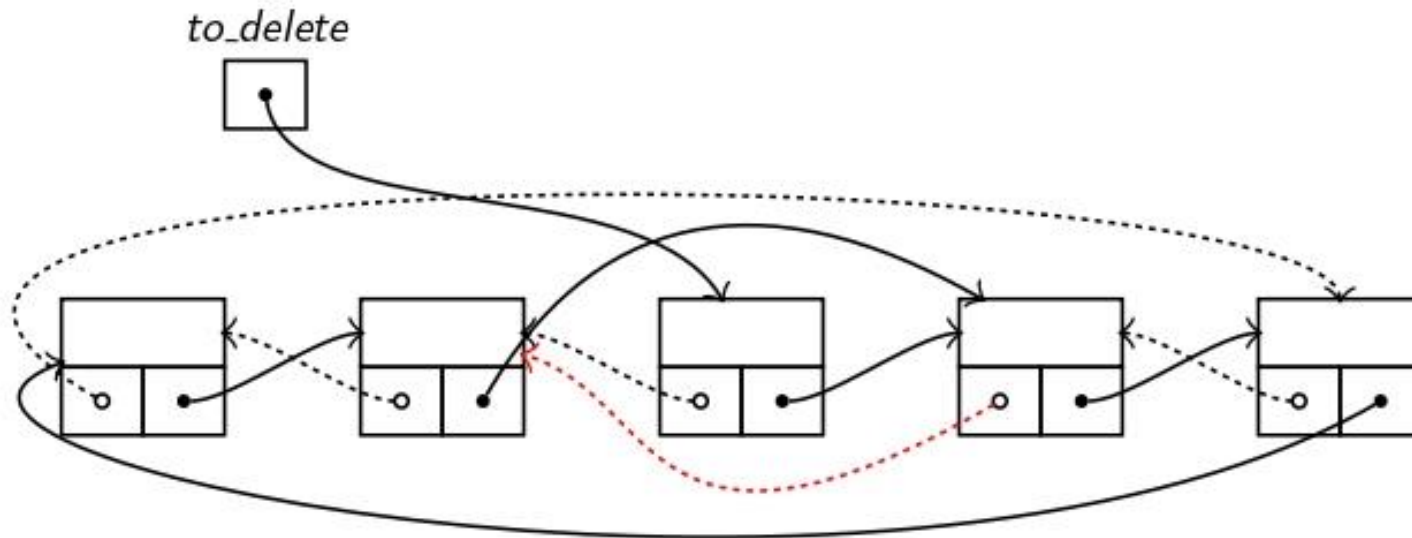
- Deleting a node from a doubly linked circular list



```
to_delete -> next -> prev = to_delete -> prev;
```

Doubly Linked Lists

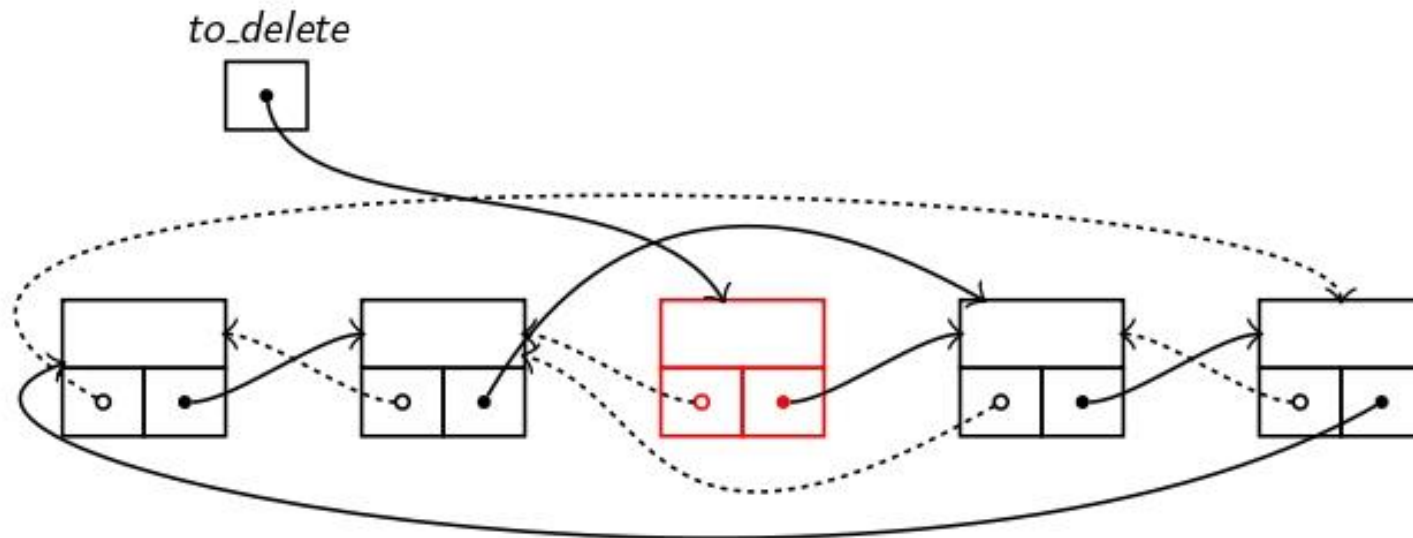
- Deleting a node from a doubly linked circular list



```
to_delete -> next -> prev = to_delete -> prev;
```

Doubly Linked Lists

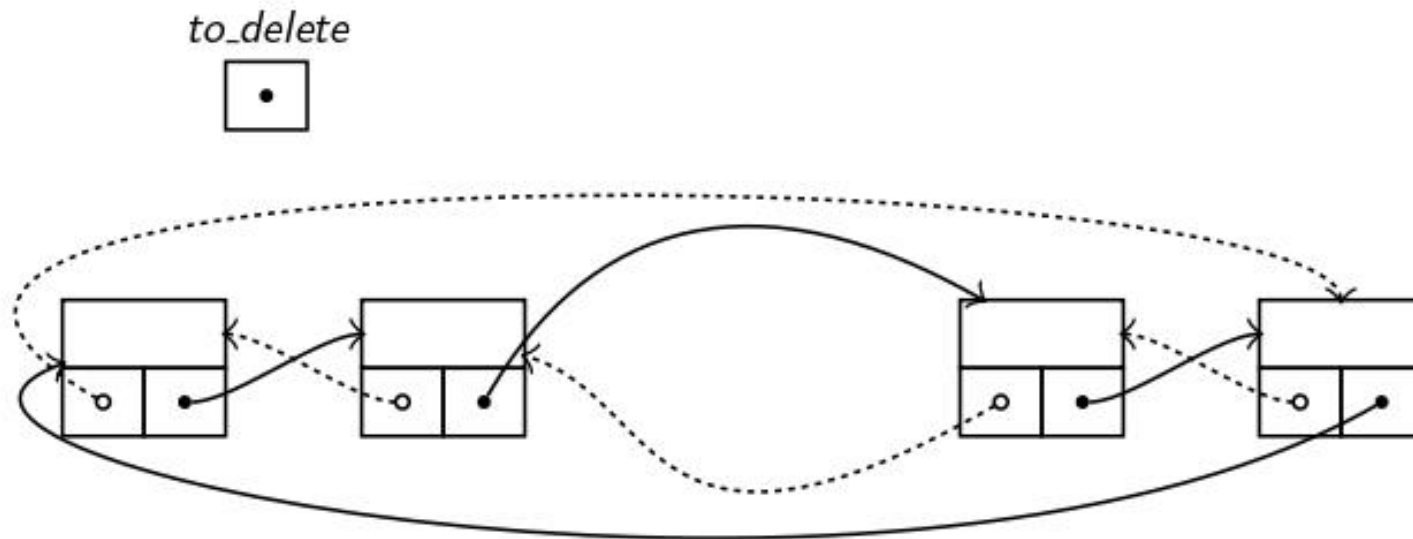
- Deleting a node from a doubly linked circular list



```
free(to_delete);
```

Doubly Linked Lists

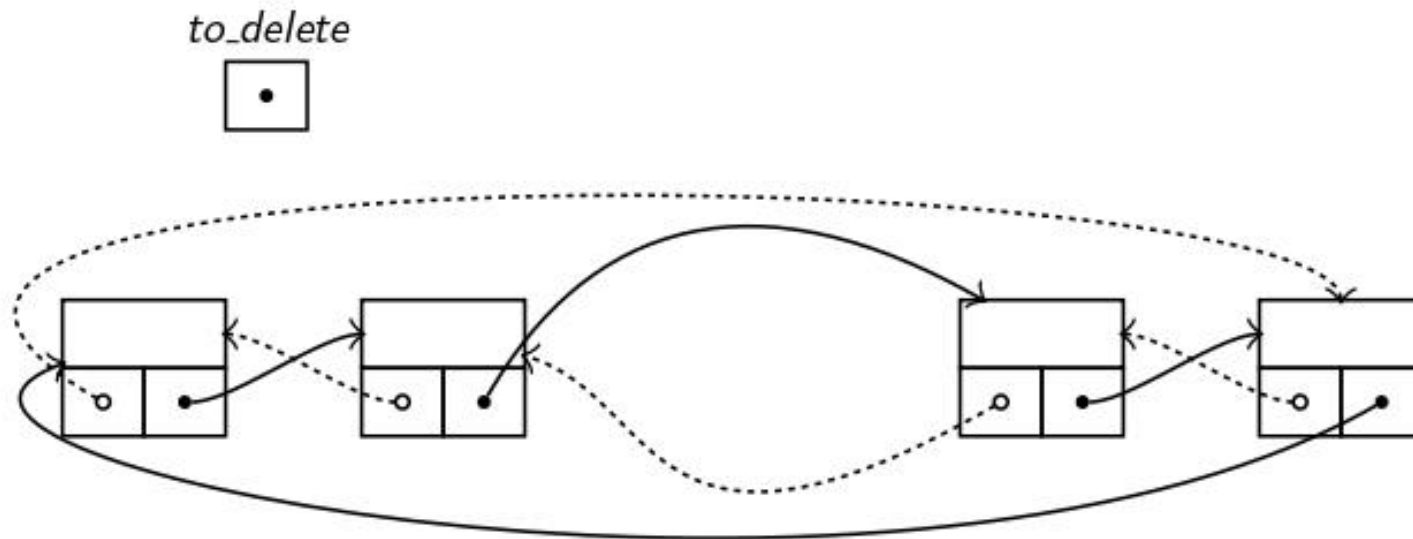
- Deleting a node from a doubly linked circular list



```
free(to_delete);
```

Doubly Linked Lists

- Deleting a node from a doubly linked circular list



Circular Doubly Linked Lists

Deleting a Node from a Circular Doubly Linked List

```
void delete(int info , node * start)
{
    /*
     * Search for the info in the list.
     */
    node *to_delete = search(info , start);
    if (to_delete) {
        /*
         * If a node was found, then just link its neighbors
         * one to another.
         */
        to_delete->prev->next = to_delete->next;
        to_delete->next->prev = to_delete->prev;
        /*
         * And, of course , free the memory used by the node.
         */
        free(to_delete);
    }
}
```




De data trecută

Pointeri – recapitulare

Transmiterea prin dublu pointer

Coada, Stiva implementate cu liste

Liste multiplu înlănțuite

Pointeri – să ne amintim

Pointerii sunt adrese în memorie către variabile.

```
int *pn;                //pointer to int  
struct div_t *pdiv;    //pointer to structure div_t
```

Accesarea valorii prin pointer:

```
double pi = 3.14159;  
double *ppi = &pi;  
printf ( "pi = %g\n" , *ppi );
```

```
*ppi = *ppi + *ppi;    //modificam valoarea de la adresa ppi  
printf ( "pi = %g\n" , pi );
```

Pointeri spre alți pointeri

Adresa stocată de un pointer este și ea tot o variabilă

Putem să folosim adresa unde este stocat un pointer – pointer către un alt pointer

```
int n = 3;
```

```
int *pn = &n;           /* pointer to n */
```

```
int **ppn = &pn;        /* pointer to address of n */
```

Pointeri spre alți pointeri

Ce face această funcție?

```
void swap ( int **a , int **b ) {  
    int *temp = *a ;  
    *a = *b ;  
    *b = temp ;  
}
```

Pointeri spre alți pointeri

Ce face această funcție?

```
void swap ( int **a , int **b ) {  
    int *temp = *a ;  
    *a = *b ;  
    *b = temp ;  
}
```

Cum putem să comparăm funcția de mai sus cu varianta standard de schimb între variabile?

```
void swap ( int *a , int *b ) {  
    int temp = *a ;  
    *a = *b ;  
    *b = temp ;  
}
```

Pointeri spre alți pointeri

```
int main()
{
    int x = 5;
    int y = 10;
    int *px = &x;
    int *py = &y;

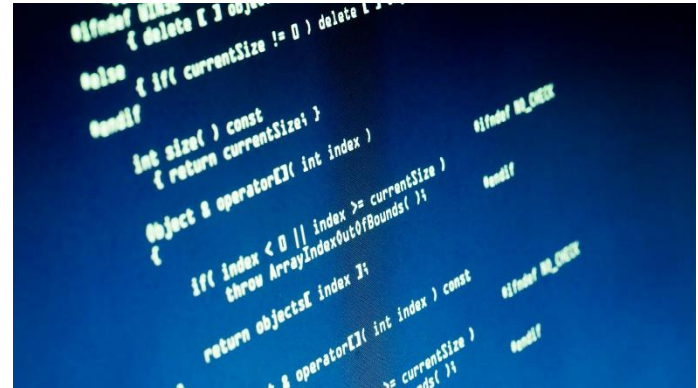
    printf("Initial values: x = %d, y = %d\n", x, y);
    swap(&px, &py);
    printf("SWAP\n");
    printf("Adresare prin nume de variabila: x = %d, y = %d\n", x, y);
    printf("Adresare prin pointer: x = %d, y = %d\n", *px, *py);
}
```

Pointeri spre alți pointeri

```
int main()
{
    int x = 5;
    int y = 10;
    int *px = &x;
    int *py = &y;
```

```
Initial values: x = 5, y = 10
SWAP
Adresare prin nume de variabila: x = 5, y = 10
Adresare prin pointer: x = 10, y = 5
```

```
printf("Initial values: x = %d, y = %d\n", x, y);
swap(&px, &py);
printf("SWAP\n");
printf("Adresare prin nume de variabila: x = %d, y = %d\n", x, y);
printf("Adresare prin pointer: x = %d, y = %d\n", *px, *py);
}
```



De data trecută –

Pointeri – recapitulare

Transmiterea prin dublu pointer

Coada, Stiva implementate cu liste

Liste multiplu înlănțuite

Moduri de lucru cu pointeri la liste

Pentru operațiile cu liste (adaugare, ștergere, modificare) avem două variante de lucru cu privire la **actualizarea pointerului spre începutul listei** în cazul în care îl modificăm în funcție:

- îl returnăm la finalul execuției funcției (lucram cu el ca simplu pointer)
- îl transmitem ca parametru prin adresa sa (lucram cu dublu pointer)

Adaugarea unui nod la început

Transmitem *pointerul* spre inceputul listei ca parametru:

```
elem * adaugaInceput(elem *lista,  
int n)  
{  
    elem *nou;  
    nou=nod_nou(n,lista);  
    lista=nou;  
    return lista;  
}
```

La apelare:

```
lista = adaugaInceput(lista, 10);
```

Adaugarea unui nod la început

Transmitem *pointerul* spre inceputul listei ca parametru:

```
elem * adaugaInceput(elem *lista,  
int n)  
{  
    elem *nou;  
    nou=nod_nou(n,lista);  
    lista=nou;  
    return lista;  
}
```

La apelare:

```
lista = adaugaInceput(lista, 10);
```

Transmitem *adresa pointerului* spre inceputul listei ca parametru:

```
void adaugaInceput(elem **plista,  
int n)  
{  
    elem *nou;  
    nou=nod_nou(n, *plista);  
    *plista=nou;  
}
```

La apelare:

```
adaugaInceput(&lista, 10);
```

Ștergerea unui nod de la început

Transmitem *pointerul* spre inceputul listei ca parametru:

```
elem *stergeInceput(elem *lista) {  
    if (lista == NULL)  
        return lista;  
    elem *p = lista;  
    lista = lista->urm;  
    free(p);  
    return lista;  
}
```

La apelare:

```
lista = stergeInceput(lista);
```

Ștergerea unui nod de la început

Transmitem *pointerul* spre inceputul listei ca parametru:

```
elem *stergeInceput(elem *lista) {  
    if (lista == NULL)  
        return lista;  
    elem *p = lista;  
    lista = lista->urm;  
    free(p);  
    return lista;  
}
```

La apelare:

lista = stergeInceput(*lista*);

Transmitem *adresa pointerului* spre inceputul listei ca parametru:

```
void stergeInceput(elem **plista) {  
    if (*plista == NULL)  
        return;  
    elem *p = *plista;  
    *plista = (*plista)->urm;  
    free(p);  
}
```

La apelare:

stergeInceput(&*lista*);

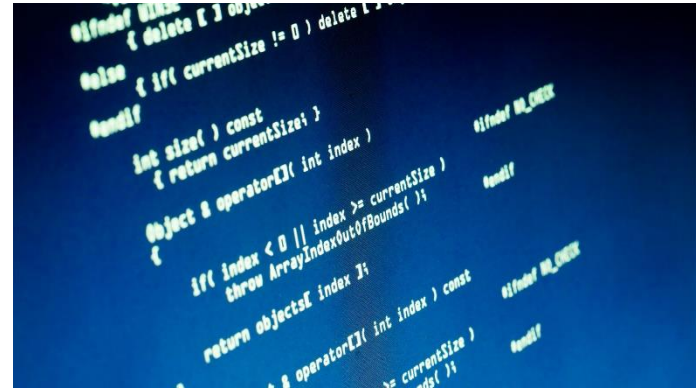
Problemă

Problemă în care **se impune transmiterea adresei pointerului** către liste:

Se citesc numere intregi dintr-un fisier text si se dauga intr-o lista simplu inlantuita.

Sa se scrie o functie care creaza 2 liste care sa contina in ordine elementele din lista initiala, una elementele pare, iar cealalta pe cele impare.

```
imparte(&lista, &lista_pare, &lista_impere);
```



De data trecută –

Pointeri – recapitulare

Transmiterea prin dublu pointer

Coadă, Stivă implementate cu liste

Liste multiplu înlănțuite

Stiva

Stiva este o structura de date LIFO (last in, first out), ceea ce inseamna ca ultimul element adaugat in stiva este primul element extras din stiva.

Operațiile fundamentale pe o stivă sunt **push** (adaugarea unui element în vârful stivei) și **pop** (scoaterea unui element din vârful stivei).

O stiva poate fi implementata in C utilizand **un vector sau o lista simpla inlantuita**. In continuare, vom parcurge cum putem implementa o stiva utilizand o listă.

Stiva

Structura unui element din stivă este aceeași cu structura unui nod la o listă liniară simplu înlănțuită:

```
struct s_listnode {  
    int element ;  
    struct s_listnode * pnext ;  
};
```

```
struct s_listnode * stack_buffer = NULL;
```

Adăugarea unui element în stivă

Adaugam un element **doar la începutul listei**.

```
void push ( int elem ) {  
    struct s_listnode *new_node = ( struct s_listnode  
    *) malloc ( sizeof ( struct s_listnode ) );  
  
    new_node->pnext = stack_buffer ;  
    new_node->element = elem ;  
    stack_buffer = new_node ;  
}
```

Ștergerea unui element din stivă

Eliminarea unui element din stivă se face **tot timpul de la început**.

```
int pop ( void ) {  
    if ( stack_buffer ) {  
        struct s_listnode *pelem = stack_buffer;  
        int elem = stack_buffer->element ;  
        stack_buffer = pelem->pnext ;  
        free ( pelem );  
        return elem ; }  
    else return 0;  
}
```

Coada

Coada este o structura de date FIFO (first in, first out), ceea ce inseamna ca primul element adaugat in coada o sa fie primul element extras din coada.

Operațiile fundamentale pe o coadă sunt **enqueue** (adaugarea unui element la capătul cozii) și **dequeue** (scoaterea unui element din capul cozii).

O coada poate fi implementata in C utilizand **un vector sau o lista simpla inlantuita**. In continuare, vom parcurge cum se poate implementa o coada utilizand o listă.

Coada

Structura unui nod al cozii este structura standard de la o listă simplu înlănțuită. E de ajutor ca pe lîndă adresa primului nod **să avem un pointer și către ultimul nod.**

```
struct s_listnode {  
float element ;  
struct s_listnode *pnext ;  
};  
  
struct s_listnode *queue_buffer = NULL;  
struct s_listnode *prear = NULL; //retinem si adresa ultimului nod
```

Adăugarea unui element în coadă

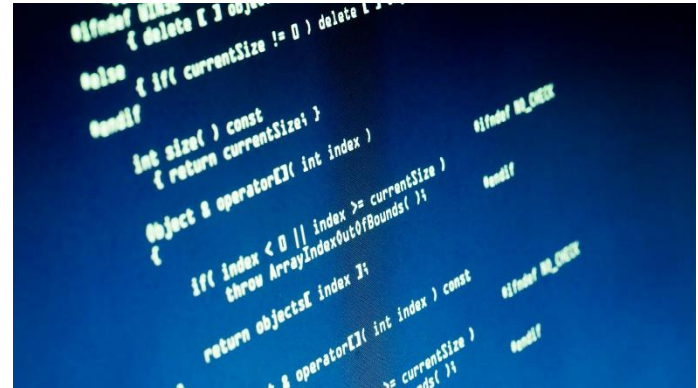
Adăugarea unui element în coadă o facem **tot timpul doar la sfârșitul listei**.

```
void enqueue ( float elem ) {  
struct s_listnode *new_node = ( struct s_listnode *) malloc (  
sizeof ( struct s_listnode ) );  
new_node->element = elem ;  
new_node->pnext = NULL; / * at rear */  
if (prear != NULL)  
    prear ->pnext = new_node ;  
else / * empty */  
    queue_buffer = new_node ;  
prear = new_node;  
}
```

Ștergerea unui element din coadă

Ștergerea unui element din coadă o vom face tot timpul de la începutul listei (cozii).

```
float dequeue ( void ) {  
if ( queue_buffer != NULL ) {  
    struct s_listnode *pelem = queue_buffer;  
    float elem = queue_buffer ->element;  
    queue_buffer = pelem->pnext ;  
    if ( pelem == prear ) / * at end */  
        prear = NULL;  
    free ( pelem );  
    return elem; }  
else  
    return 0; }
```



De data trecută –

Pointeri – recapitulare

Transmiterea prin dublu pointer

Coadă, Stiva implementate cu liste

Liste multiplu înlănțuite

Liste multiplu înlănțuite - Multiliste

Sunt cazuri în care informația dintr-o listă e necesară la un moment dat **într-o anumită ordine**, iar la alt moment **în altă ordine**.

De exemplu dacă avem o bază de date cu studenții facultății, am avea nevoie de **lista studenților ordonată alfabetic** sau **ordonată descrescător în funcție de medie**.

Ce variante am avea?

- **Să reordonăm** lista când alfabetic, când după medie (mult timp consumat)
- Să reținem studenții în **2 liste ordonate diferit** (memorie duplicată, multă memorie folosită)

Liste multiplu înlănțuite - Multiliste

Implementarea evidenței sub formă de listă simplu înlănțuită ordonată după nume sau ca listă simplu înlănțuită ordonată după medie este nesatisfăcătoare, deoarece s-ar impune **o operație de reordonare a listei** de fiecare dată când se comandă afișarea după celălalt criteriu.

Dacă informațiile ar fi păstrate în 2 liste ordonate independente, **redundanța informațiilor complică operațiile** de adăugare și ștergere a unei persoane din evidență (aceste operații ar trebui făcute pe ambele liste).

Liste multiplu înlănțuite - Multiliste

O soluție este ca persoanele să fie păstrate într-o **listă multiplu înlănțuită**.

Această listă are în noduri informațiile despre persoane, **noduri care sunt înlănțuite simultan în două liste ordonate**, una după nume și una după medie.

Aceleași noduri fizice fac parte simultan din ambele liste, informația despre fiecare persoană apare o singură dată.

Liste multiplu înlănțuite - Multiliste

Se definește tipul structurat *student* pentru tipul nodurilor listei. Pe lângă câmpurile de date (*nume*, *medie*), **aceasta structură conține două câmpuri de înlănțuire (pointeri): *urm_nume* și *urm_medie*.**

Câmpul *urm_nume* indică adresa următorului nod care are numele alfabetic după numele din nodul curent, iar câmpul *urm_medie* indică următorul nod care are media mai mare decât media din nodul curent.

Pentru accesul la structura de date, **sunt necesari doi pointeri spre cele două începuturi ale listei** - un pointer spre capul listei ordonată după nume, în variabila globală *lista_nume*, și un pointer spre capul listei după medie în variabila globală *lista_medie*.

Structura unui nod

```
typedef struct student{  
    char *nume;  
    float medie;  
    struct student *urm_nume;  
    struct student *urm_varsta;  
} persoana;  
  
student *lista_nume =NULL, *lista_medie =NULL;
```

Căutarea în listă

Dintre operațiile pe structura de date corespunzătoare evidenței studenților, operațiile de **căutare și afișare se efectuează ca operații obișnuite pe liste simplu înlănțuite.**

```
student *cauta( char *n) {  
    student *p;  
    for( p = lista_nume; p !=NULL ; p =p->urm_nume)  
        if( strcmp(p->nume,n ) == 0)  
            return p;  
    return NULL;  
}
```

Afişarea listei

```
void afis_num( void ) {  
    student *p;  
    for( p =lista_num; p != NULL ; p =p->urm_num)  
        printf("%s %f\n",p->nume, p->medie);  
}
```

```
void afis_medie( void ) {  
    student *p;  
    for( p =lista_medie; p != NULL ; p =p->urm_medie)  
        printf("%s %f\n",p->nume, p->medie);  
}
```

Adăugarea în listă

```
void adauga(char *n, float m) {  
    student *t;  
    if(( t = cauta( n )) != NULL) {  
        printf("Eroare: %s exista deja \n", n);  
        return; }  
    else  
        if(( t = (persoana*)malloc(sizeof(persoana))) == NULL ||  
           ( t->nume = (char*)malloc(strlen(n)+1)) == NULL ) {  
            printf("Eroare : memorie insuficienta\n"); exit( 1 ); }  
        else {  
            strcpy( t->nume,n);  
            t->medie = m;  
            adauga_nume(t);  
            adauga_medie(t); }  
}
```


Adăugarea în listă

Cele două funcții, *adauga_nume* și *adauga_medie*, realizează înlănțuirile nodului nou în cele 2 liste, *lista_varsta* respectiv *lista_nume*. Fiecare dintre aceste două funcții este o operație simplă de inserție a unui nou nod în lista ordonată respectivă.

```
void adauga_nume(student *nou) {  
    student *q1, *q2;  
    for(q1=q2=lista_nume; q1!=NULL && strcmp(q1->nume, nou->nume)<0; q2=q1, q1=q1->urm_nume);  
    nou->urm_nume = q1;  
    if(q1 == q2 )  
        lista_nume=nou;  
    else  
        q2->urm_nume = nou;  
}
```

Adăugarea în listă

Cele două funcții, *adauga_ume* și *adauga_medie*, realizează înlănțuirile nodului nou în cele 2 liste, *lista_varsta* respectiv *lista_ume*. Fiecare dintre aceste două funcții este o operație simplă de inserție a unui nou nod în lista ordonată respectivă.

```
void adauga_medie( student *nou) {  
    student *q1,*q2;  
    for(q1=q2=lista_medie; q1!=NULL && q1->medie<nou->medie;  
        q2=q1,q1=q1->urm_medie);  
    nou->urm_medie = q1;  
    if(q1 == q2)  
        lista_medie=nou;  
    else  
        q2->urm_medie = nou;  
}
```

Ștergerea din listă

```
void elimin( char *s){
    scot_ume(s);
    scot_medie(s);
}

void scot_ume( char *l) {
    student *q1,*q2;
    for(q1=q2=lista_ume; q1!= NULL && strcmp( q1->ume,l)<0; q2=q1,
        q1=q1->urm_ume);
    if( q1 != NULL && strcmp (q1->ume,l) ==0)
        if(q1 == q2 )
            lista_ume=lista_ume->urm_ume;
        else
            q2->urm_ume = q1->urm_ume;
    else
        printf(" Eroare: %s nu apare in evidenta\n",l);
}
```

Ștergerea din listă

```
void scot_medie( char *l) {  
    student *q1,*q2;  
    for(q1=q2=lista_medie; q1!= NULL && strcmp( q1-  
        >nume,l)!=0; q2=q1, q1=q1->urm_medie);  
  
    if( q1 != NULL && strcmp (q1->nume,l) ==0)  
        if(q1 == q2 )  
            lista_medie= lista_medie->urm_medie;  
        else  
            q2->urm_medie = q1->urm_medie;  
    else  
        printf(" Eroare: %s nu apare in evidenta\n",l);  
}
```

```
if (delete [ ] object)
{ delete [ ] object; }
else
{ if (currentSize != 0) delete [ ] object; }
endif

int size() const
{ return currentSize; }

Object & operator[] (int index)
{
    if (index < 0 || index >= currentSize)
        throw ArrayIndexOutOfBoundsException();
    return objects[index];
}

Object & operator[] (int index) const
{
    if (index < 0 || index >= currentSize)
        throw ArrayIndexOutOfBoundsException();
    return objects[index];
}
```

Vă mulțumesc!